

Intro Web + Django + ORM · Laboratorio de Programación y Lenguajes · UNTDF 2026

Esta guía está pensada para alumnos que no pudieron estar en clase, para quienes necesitan repasar con más profundidad, y para los que cursan offline. Contiene explicaciones ampliadas, tutoriales paso a paso, FAQ, anti-patrones, cheatsheets y referencias a la documentación oficial.

Cómo leer esta guía:

- Si asististe a las 3 clases, podés ir directo a §11 (FAQ) y §12 (Checklist TP-4).
- Si no asististe, leela **secuencialmente** — está construida para reemplazar la clase.
- Los ejemplos son **auto-contenidos**: podés ejecutarlos sin tener el proyecto abierto.

1. ¿Qué vas a aprender?

Al terminar este tema podés:

1. Explicar qué es una app web y diferenciarla de un sitio estático.
2. Identificar las 3 capas (presentación/negocio/datos) en cualquier sistema.
3. Clasificar responsabilidades usando el patrón **MVC** (y su variante **MVT** de Django).
4. Interpretar peticiones y respuestas HTTP, incluyendo métodos y códigos.
5. Instalar Django, crear proyecto+app y levantar un servidor de desarrollo.
6. Escribir **class-based views** (porque en esta cátedra NO usamos funciones como vistas).
7. Modelar entidades con **Django ORM**: campos, relaciones FK/M2M, `null / blank`, `on_delete`.
8. Generar y aplicar **migraciones** (`makemigrations` + `migrate`).
9. Consultar la base con **QuerySet**: `filter`, `annotate`, `Q`, `F`, `order_by`, `slicing`.
10. Resolver al 100% el **TP-4** (los 4 modelos + las 4 queries + tests verdes).

2. Parte I — Introducción a la programación web

2.1. Internet, Web y HTTP — el stack base

Internet es la red física de computadoras interconectadas que usa TCP/IP (capas 3 y 4 del modelo OSI). **La Web** es una **aplicación** que corre sobre Internet usando el protocolo **HTTP**.

Analogía: Internet es la red de rutas; la Web es el sistema de entregas que usa esas rutas.

HTTP funciona con un modelo **petición/respuesta**:

- El **cliente** (navegador, curl, app móvil) envía una **request**.
- El **servidor** (tu código Django en producción) procesa y devuelve una **response**.

Cada interacción es **sin estado** (stateless): el servidor, por defecto, no recuerda entre dos peticiones del mismo cliente. Para mantener estado se usan cookies + sesiones (Tema 06).

2.2. Sitio web vs aplicación web

Característica	Sitio estático	App web
HTML	Mismo para todos	Generado según usuario/estado
Backend	Archivos servidos	Proceso que ejecuta lógica
Base de datos	Suele no tener	Casi siempre tiene
Ejemplo	Página de tesis alojada en GitHub Pages	SIU-Guaraní, Moodle, Instagram

Regla práctica: si al usuario le aparece algo diferente cuando se loguea, es una **app**.

2.3. Arquitectura en 3 capas

La arquitectura más común para apps web separa el código en **tres capas** con responsabilidades claras:

- **Capa de presentación** — lo que ve el usuario: HTML, CSS, JS. En Django: **templates**.
- **Capa de negocio (aplicación)** — reglas del dominio, orquestación. En Django: **views.py** (CBVs).
- **Capa de datos (persistencia)** — guardar y recuperar. En Django: **models.py** + ORM + SQLite/PostgreSQL.

¿Por qué separar? Tres razones muy concretas:

1. **Cambiar la BD** (de SQLite a PostgreSQL) **no toca** las pantallas.
2. **Cambiar el diseño** (rediseñar HTML) **no toca** la lógica.
3. **Testear** cada capa por separado es más fácil (los tests del TP-4 solo prueban capa de datos).

2.4. Patrón MVC / MVT

El patrón **Model-View-Controller** asigna responsabilidades a 3 objetos:

- **Model:** los datos (estructura + reglas de dominio).
- **View:** la presentación (HTML que ve el usuario).
- **Controller:** el "pegamento" — recibe la request, pide datos al Model, elige la View, devuelve la response.

Django usa una terminología ligeramente distinta llamada **MVT** (Model-View-Template):

MVC clásico	MVT Django	Archivo Django
Model	Model	<code>models.py</code>
View (HTML)	Template	<code>templates/...html</code>
Controller	View	<code>views.py</code> ← trampa de nombre

Trampa de nombres: cuando escribimos `class LibroDetailView(DetailView)` en `views.py`, estamos escribiendo lo que el MVC clásico llama **controlador**. Es solo nomenclatura.

2.5. HTTP en detalle

Estructura de una request:

```
GET /catalogo/libros/42/ HTTP/1.1
Host: biblioteca.untdf.edu.ar
User-Agent: Mozilla/5.0 ...
Accept: text/html
Cookie: sessionid=abc123

(cuerpo vacío en GET)
```

Estructura de una response:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1547
Set-Cookie: csrftoken=...

<!DOCTYPE html>
<html>...
```

Métodos que vas a usar:

- **GET** — leer, sin efectos secundarios. Cacheable, idempotente.
- **POST** — crear o enviar datos que modifican estado.
- **PUT/PATCH/DELETE** — se ven en Tema 05 cuando implementes ModelForms.

Códigos de estado — los de este tema:

- **2xx** éxito (200 OK, 201 Created, 204 No Content)
- **3xx** redirección (301 Moved, 302 Found)
- **4xx** error del cliente (400 Bad Request, 403 Forbidden, 404 Not Found)
- **5xx** error del servidor (500 Internal Server Error)

Ejercicio autónomo — abre una terminal y corre:

```
curl -i https://httpbin.org/get
curl -i https://httpbin.org/status/404
curl -i -X POST https://httpbin.org/post -d "libro=Sapiens&autor=Harari"
```

Anotá el status, el Content-Type y los primeros 3 headers de cada respuesta.

3. Parte II — Frameworks web

3.1. Framework vs librería

Librería: vos la llamás cuando la necesitás.

```
import requests
response = requests.get("https://api.ejemplo.com")
```

Vos controlás el flujo.

Framework: el framework te llama a vos.

```
# Vos escribís esto
class LibroListView(ListView):
    model = Libro

# Django internamente hace:
# 1. Cuando llega GET /libros/, busca la ruta en urls.py
# 2. Encuentra LibroListView, llama a .as_view()
# 3. Llama a tu get_queryset() si lo definiste
# 4. Renderiza el template
# 5. Te devuelve la HttpResponse
```

El framework controla el flujo. Vos “llenás los huecos”.

Esto se llama **inversión de control** o **principio de Hollywood** (“*don’t call us, we’ll call you*”).

3.2. ¿Por qué Django?

- **Maduro** (2005) y con versiones LTS estables.
- **Baterías incluidas:** ORM, migraciones, admin, auth, sessions, i18n, forms — todo integrado.
- **Seguridad por defecto:** protección CSRF, XSS, SQL injection — out of the box.
- **Ecosistema grande:** miles de paquetes third-party (Django REST Framework, django-allauth, etc).

Cuándo NO Django:

- APIs muy simples → FastAPI (se ve al final del año).
 - Microservicios minimalistas → Flask.
 - Sitios estáticos → Jekyll/Hugo.
-

4. Parte III — Django paso a paso (reemplazo de Clase 2)

4.1. Instalación

Requisitos: Python 3.13+ instalado y `python --version` funcionando en terminal.

```
# 1. Crear una carpeta para el proyecto
mkdir biblioteca-untdf
cd biblioteca-untdf

# 2. Crear entorno virtual
python -m venv .venv

# 3. Activar
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# Si PowerShell se queja: Set-ExecutionPolicy -Scope Process RemoteSigned
# Linux/Mac:
source .venv/bin/activate

# 4. Instalar Django (el TP-4 pide 5.1+)
pip install "django>=5.1,<6.0"

# 5. Verificar
django-admin --version
# Debería imprimir: 5.1.x o similar
```

¿Por qué un `venv`? Si instalás Django globalmente, tu próximo proyecto puede pisarle la versión. El `venv` lo aísla. Regla: un `venv` por proyecto.

4.2. Crear proyecto y app

```
# Crear el "proyecto" biblioteca en la carpeta actual
django-admin startproject biblioteca .

# Notar el punto al final – si lo omitís, crea biblioteca/biblioteca/...

# Crear la "app" catalogo (una feature coherente)
python manage.py startapp catalogo

# Aplicar migraciones iniciales (tablas de auth, sessions, etc)
python manage.py migrate

# Levantar el servidor
python manage.py runserver
```

Abrió `http://127.0.0.1:8000/` en el navegador. Deberías ver la página de bienvenida de Django.

Terminología importante:

- **Proyecto** = configuración global (settings, urls raíz, wsgi). En nuestro caso: `biblioteca/`.
- **App** = una feature coherente, reutilizable, con sus propios models/views/urls. En nuestro caso: `catalogo/`.
- Un proyecto tiene 1...N apps.

4.3. Estructura generada

```
biblioteca-untdf/  
├── .venv/  
├── manage.py           ← CLI: runserver, migrate, test, shell, ...  
├── db.sqlite3          ← BD local (se crea tras migrate)  
├── biblioteca/  
│   ├── __init__.py     ← "proyecto"  
│   ├── settings.py     ← configuración central  
│   ├── urls.py         ← URLs raíz (router global)  
│   ├── wsgi.py         ← servidor prod (sync)  
│   └── asgi.py         ← servidor prod (async)  
├── catalogo/  
│   ├── __init__.py     ← "app"  
│   ├── admin.py        ← registro del modelo en el admin generado  
│   ├── apps.py         ← config de la app  
│   ├── migrations/  
│   │   └── __init__.py ← archivos de migración  
│   ├── models.py       ← datos (TP-4: 4 modelos)  
│   ├── tests.py        ← tests  
│   └── views.py        ← controladores (CBVs)
```

Archivos **nuevos** que vas a crear (no vienen generados):

- `catalogo/urls.py` — URLs de la app
- `catalogo/templates/catalogo/*.html` — templates
- `catalogo/queries.py` — las 4 queries del TP-4

4.4. `settings.py` — qué tocar

```
# biblioteca/settings.py (extracto)  
  
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "catalogo", # ← OBLIGATORIO agregar tu app  
]  
  
DATABASES = {  
    "default": {  
        "ENGINE": "django.db.backends.sqlite3",  
        "NAME": BASE_DIR / "db.sqlite3",  
    }  
}  
  
LANGUAGE_CODE = "es-ar"  
TIME_ZONE = "America/Argentina/Ushuaia"
```

Error más frecuente: olvidar agregar `"catalogo"` a `INSTALLED_APPS` → `makemigrations` no detecta tus modelos.

4.5. URLconf — el router

```
# biblioteca/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("catalogo/", include("catalogo.urls")),
]
```

```
# catalogo/urls.py (CREAR este archivo nuevo)
from django.urls import path
from .views import HolaMundoView

app_name = "catalogo"

urlpatterns = [
    path("hola/", HolaMundoView.as_view(), name="hola"),
]
```

Regla de cátedra: todas las rutas usan `.as_view()` porque todas las vistas son clases.

4.6. Primera class-based view — TemplateView

```
# catalogo/views.py
from django.views.generic import TemplateView

class HolaMundoView(TemplateView):
    """Primera CBV del curso – muestra un saludo personalizable."""

    template_name = "catalogo/hola.html"

    def get_context_data(self, **kwargs):
        ctx = super().get_context_data(**kwargs)
        ctx["mensaje"] = "Hola 3º año Sistemas UNTDF"
        ctx["anio"] = 2026
        return ctx
```

```
<!-- catalogo/templates/catalogo/hola.html -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Hola Django</title>
</head>
<body>
    <h1>{{ mensaje }}</h1>
    <p>Año académico: {{ anio }}</p>
</body>
</html>
```

Ruta exacta del template: `catalogo/templates/catalogo/hola.html` — sí, “catalogo” aparece dos veces. Es convención: el sistema de templates busca en todas las apps, y anidar en una subcarpeta con el nombre de la app evita colisiones con otras apps que tengan un `hola.html`.

Probá: `http://127.0.0.1:8000/catalogo/hola/`

4.7. ¿Por qué CBV y no FBV?

FBV (PROHIBIDO en esta cátedra)	CBV (estándar cátedra)
<code>def home(request): return render(...)</code>	<code>class HomeView(TemplateView): ...</code>
<code>@login_required</code> (decorador)	<code>LoginRequiredMixin</code>
Lógica mezclada	Métodos sobrescribibles

Razones de la decisión:

1. **Coherencia con la cursada:** venís del Módulo I con POO.
2. **Reuso vía mixins:** `LoginRequiredMixin`, `UserPassesTestMixin`, etc.
3. **Estructura clara:** `get_context_data`, `get_queryset`, `form_valid`, `dispatch` — puntos explícitos de extensión.
4. **Estándar profesional:** proyectos grandes de Django usan CBVs.

4.8. Jerarquía de CBVs útiles

- **View** — raíz de todo.
- **TemplateView** — mostrar un HTML con contexto (la que usamos hoy).
- **RedirectView** — redirigir.
- **ListView** — listar objetos de un modelo (Tema 05).
- **DetailView** — ver uno solo (Tema 05).
- **CreateView / UpdateView / DeleteView** — ABM con ModelForms (Tema 05).

Para el TP-4 no necesitás vistas. Pero vas a usar CBVs desde el primer ejercicio post-TP.

4.9. Tutorial A — CRUD mínimo de juguete (opcional, 15 min)

Supongamos que querés listar todos los autores con una `ListView`:

```
# catalogo/views.py
from django.views.generic import ListView
from .models import Autor

class AutorListView(ListView):
    model = Autor
    template_name = "catalogo/autor_list.html"
    context_object_name = "autores"
```

```
<!-- catalogo/templates/catalogo/autor_list.html -->
<h1>Autores</h1>
<ul>
{% for autor in autores %}
  <li>{{ autor.nombre }} ({{ autor.email }})</li>
{% empty %}
  <li>No hay autores aún.</li>
{% endfor %}
</ul>
```

```
# catalogo/urls.py
from .views import HolaMundoView, AutorListView

urlpatterns = [
    path("hola/", HolaMundoView.as_view(), name="hola"),
    path("autores/", AutorListView.as_view(), name="autor-list"),
]
```

Abrí `/catalogo/autores/`. Si no hay autores, muestra el mensaje empty. Esto lo verás completo en Tema 05.

5. Parte IV — Django ORM (reemplazo de Clase 3)

5.1. Persistencia — el problema de fondo

En memoria tenés **objetos** con identidad, métodos y relaciones navegables:

```
libro = Libro("Sapiens", autor=harari)
libro.autor.email # → "harari@example.com"
```

En disco tenés **tablas** con filas, columnas y relaciones por clave foránea:

```
libro: id | titulo | autor_id
      1 | Sapiens | 17

autor: id | nombre | email
      17 | Harari | harari@example.com
```

Impedance mismatch: los dos mundos no encajan naturalmente:

Mundo OO	Mundo Relacional	Problema
Objetos con métodos	Filas pasivas	SQL no tiene comportamiento
Herencia	Sin herencia nativa	Hay que mapear (tabla por clase, etc)
Colecciones (<code>libro.categorias</code>)	Tabla intermedia M2M	El ORM la maneja por vos
Identidad por <code>==</code> / hash	Identidad por PK	<code>==</code> lógico vs <code>is</code>
Navegación libre	JOINS explícitos	N+1 es el precio de la ilusión

El **ORM** (Object-Relational Mapper) es una capa que traduce entre los dos mundos, **pero la traducción tiene costos** — de ahí que entender SQL siga siendo necesario.

5.2. Alternativas en Python

Solución	Uso	Comentario
Django ORM	Django apps	Nuestro foco — integrado, migraciones incluidas
SQLAlchemy	Python general	Más flexible, más complejo
Peewee	Scripts pequeños	Minimalista
pickle, shelve	Caché local	No para producción multiusuario
SQL crudo	Control total	Más código, menos portable

5.3. Modelos Django — el TP-4

Definamos los 4 modelos del TP-4. Abrí `tp-repo/catalogo/models.py` y reemplazá los `pass` :

```

# catalogo/models.py
from django.db import models

class Autor(models.Model):
    nombre = models.CharField(max_length=120)
    email = models.EmailField(unique=True)
    biografia = models.TextField(blank=True)

    def __str__(self) -> str:
        return self.nombre

class Categoria(models.Model):
    nombre = models.CharField(max_length=80, unique=True)

    def __str__(self) -> str:
        return self.nombre

class Libro(models.Model):
    titulo = models.CharField(max_length=200)
    isbn = models.CharField(max_length=32, unique=True)
    fecha_publicacion = models.DateField()
    cantidad_total = models.PositiveIntegerField(default=1)

    autor = models.ForeignKey(
        Autor,
        on_delete=models.PROTECT,
        related_name="libros",
    )
    categorias = models.ManyToManyField(
        Categoria,
        related_name="libros",
        blank=True,
    )

    def prestamos_activos(self) -> int:
        return self.prestamos.filter(fecha_devolucion__isnull=True).count()

    def disponibles(self) -> int:
        return self.cantidad_total - self.prestamos_activos()

    def tiene_disponibles(self) -> bool:
        return self.disponibles() > 0

    def __str__(self) -> str:
        return self.titulo

class Prestamo(models.Model):
    libro = models.ForeignKey(
        Libro,
        on_delete=models.CASCADE,
        related_name="prestamos",
    )
    nombre_prestatario = models.CharField(max_length=120)
    fecha_prestamo = models.DateField()
    fecha_devolucion = models.DateField(null=True, blank=True)

    def esta_activo(self) -> bool:
        return self.fecha_devolucion is None

    def __str__(self) -> str:
        estado = "activo" if self.esta_activo() else "devuelto"
        return f"{self.libro.titulo} - {self.nombre_prestatario} ({estado})"

```

5.4. Campos comunes que vas a usar

Campo	Uso	Parámetros típicos
<code>CharField</code>	Texto corto	<code>max_length=</code> obligatorio
<code>TextField</code>	Texto largo	<code>blank=True</code> si opcional
<code>EmailField</code>	Email	Valida formato
<code>DateField</code>	Fecha (año-mes-día)	<code>null=True</code> , <code>blank=True</code> si opcional
<code>DateTimeField</code>	Fecha + hora	<code>auto_now_add=True</code> , <code>auto_now=True</code>
<code>PositiveIntegerField</code>	Entero ≥ 0	<code>default=</code>
<code>IntegerField</code>	Entero cualquiera	
<code>BooleanField</code>	True/False	<code>default=False</code>
<code>URLField</code>	URL	
<code>SlugField</code>	Texto URL-safe	<code>unique=True</code>
<code>DecimalField</code>	Decimal exacto (precio)	<code>max_digits</code> , <code>decimal_places</code>
<code>FileField/ImageField</code>	Archivos	<code>upload_to=</code>

5.5. `null=True` vs `blank=True` — el clásico que cae en parcial

Parámetro	Qué controla	Dónde vive
<code>null=True</code>	Permite <code>NULL</code> en la base de datos	Nivel BD
<code>blank=True</code>	Permite vacío en formularios y validación	Nivel Django

Regla práctica:

- **Campos de texto** (`CharField` , `TextField` , `EmailField`): **solo** `blank=True` . Django usa `""` en vez de `NULL` para strings vacíos — convención.
- **Campos no-texto opcionales** (`DateField` , `IntegerField`): **ambos** `null=True`, `blank=True` .

En el TP-4:

- `biografia = TextField(blank=True)` — texto, solo blank.
- `fecha_devolucion = DateField(null=True, blank=True)` — fecha, ambos.

5.6. Relaciones

ForeignKey (N:1) — `Libro` tiene un `Autor` , un autor tiene **varios** libros:

```
autor = models.ForeignKey(Autor, on_delete=models.PROTECT, related_name="libros")
```

- `on_delete=` — OBLIGATORIO desde Django 2.x.
- `related_name="libros"` — qué nombre usar para navegar de Autor a sus Libros:
`autor.libros.all()` . Si no lo ponés, Django usa `autor.libro_set.all()` .

ManyToManyField (N:M) — `Libro` puede tener **varias** `Categoria` , una categoría tiene **varios** libros:

```
categorias = models.ManyToManyField(Categoria, related_name="libros", blank=True)
```

Django crea automáticamente una tabla intermedia `catalogo_libro_categorias` . **No la tocás.**

OneToOneField (1:1) — no se usa en el TP-4, pero está bueno conocerlo: extender `User` con un perfil.

5.7. `on_delete` — estrategias

Valor	Efecto	Cuándo
<code>CASCADE</code>	Borra los hijos	Prestamo → Libro (TP-4)
<code>PROTECT</code>	Prohíbe borrar si hay hijos	Libro → Autor (TP-4)
<code>SET_NULL</code>	Pone NULL en hijos (requiere <code>null=True</code>)	"autor desconocido"
<code>SET_DEFAULT</code>	Pone default (requiere <code>default=</code>)	Autor "Anónimo"
<code>SET(func)</code>	Pone valor calculado	
<code>DO_NOTHING</code>	Django no hace nada	Control manual

Justificación de los `on_delete` del TP-4:

- `Libro → Autor = PROTECT` : la biblioteca **no debe** borrar un autor que todavía tiene libros en el catálogo — sería corromper el catálogo bibliográfico.
- `Prestamo → Libro = CASCADE` : si se descarta un libro del catálogo, sus registros de préstamo histórico desaparecen con él (decisión del TP — en una biblioteca real querías mantener el historial, usarías `SET_NULL`).

5.8. Migraciones

```
# 1. Generar archivo de migración (NO toca la BD)
python manage.py makemigrations catalogo

# Output:
# Migrations for 'catalogo':
#   catalogo/migrations/0001_initial.py
#       - Create model Autor
#       - Create model Categoria
#       - Create model Libro
#       - Create model Prestamo

# 2. Ver el SQL que se va a ejecutar (didáctico)
python manage.py sqlmigrate catalogo 0001

# 3. Aplicar a la BD
python manage.py migrate

# 4. Si cambiás un modelo, repetir 1 + 3
python manage.py makemigrations
python manage.py migrate
```

Reglas clave:

1. `makemigrations` **genera un archivo**, no toca la BD.
2. `migrate` **aplica** los archivos pendientes.
3. **NO editar** migraciones ya aplicadas. Si te equivocaste, generá otra migración que corrija.
4. Las migraciones **se commitean** al repositorio — son parte del código.

Reset de emergencia (solo en desarrollo):

```
rm db.sqlite3
rm catalogo/migrations/0*.py # todos menos __init__.py
python manage.py makemigrations catalogo
python manage.py migrate
```

5.9. CRUD con QuerySet

Entrá al shell: `python manage.py shell` .

```

>>> from catalogo.models import Autor, Categoria, Libro, Prestamo
>>> from datetime import date

# CREATE
>>> ursula = Autor.objects.create(
...     nombre="Ursula K. Le Guin",
...     email="ursula@example.com",
...     biografia="Autora de SF y fantasía.",
... )

>>> sf = Categoria.objects.create(nombre="ciencia ficción")
>>> fant = Categoria.objects.create(nombre="fantasía")

>>> libro = Libro.objects.create(
...     titulo="Los desposeídos",
...     isbn="978-0000000001",
...     fecha_publicacion=date(1974, 1, 1),
...     cantidad_total=2,
...     autor=ursula,
... )
>>> libro.categorias.add(sf, fant) # M2M: add/remove/set/clear

# READ
>>> Libro.objects.all()
<QuerySet [ <Libro: Los desposeídos>]>

>>> Libro.objects.get(isbn="978-0000000001") # get = 1 o excepción
<Libro: Los desposeídos>

>>> Libro.objects.filter(autor=ursula) # QuerySet (puede ser 0, 1, N)
<QuerySet [ <Libro: Los desposeídos>]>

>>> Libro.objects.filter(autor__nombre__icontains="le guin")
# lookups: double underscore navega relaciones

# UPDATE (individual)
>>> libro.cantidad_total = 3
>>> libro.save()

# UPDATE (masivo - 1 query SQL)
>>> from django.db.models import F
>>> Libro.objects.filter(autor=ursula).update(
...     cantidad_total=F("cantidad_total") + 1
... )
1

# DELETE
>>> libro.delete()

# QuerySet encadenado
>>> Libro.objects.filter(cantidad_total__gt=0).exclude(categorias__nombre="obsoleta").order_by("--fecha_publicacion")[:5]

```

Claves para entender QuerySet:

- Es **lazy**: no hace SQL hasta que iterás, sliceás, contás o llamás `list()`.
- Es **encadenable**: `filter().exclude().order_by()` devuelve QuerySet.
- Se **cachea** por instancia: la primera vez que iterás, se cachea; la segunda vez reutiliza.
- **count()** vs **len()**: usar `.count()` — evita cargar todo a memoria.
- **get()** vs **filter().first()**: `get()` lanza `DoesNotExist` o `MultipleObjectsReturned`; `filter().first()` devuelve `None` si no hay.

5.10. Lookups ORM — cheatsheet

Lookup	SQL	Ejemplo
<code>__exact</code>	<code>= valor</code>	<code>filter(nombre__exact="Ursula")</code>
<code>__iexact</code>	<code>= valor case-insensitive</code>	<code>filter(email__iexact="U@E.com")</code>
<code>__contains</code>	<code>LIKE %valor%</code>	<code>filter(titulo__contains="desp")</code>
<code>__icontains</code>	<code>LIKE case-insensitive</code>	<code>filter(titulo__icontains="SAPI")</code>
<code>__startswith</code> / <code>__endswith</code>	<code>LIKE valor% / %valor</code>	
<code>__gt</code> , <code>__gte</code> , <code>__lt</code> , <code>__lte</code>	<code>> >= < <=</code>	<code>filter(cantidad_total__gt=0)</code>
<code>__in</code>	<code>IN (...)</code>	<code>filter(id__in=[1,2,3])</code>
<code>__range</code>	<code>BETWEEN</code>	<code>filter(fecha__range=(d1, d2))</code>
<code>__isnull</code>	<code>IS NULL/NOT NULL</code>	<code>filter(fecha_devolucion__isnull=True)</code>
<code>__date</code> , <code>__year</code> , <code>__month</code> , <code>__day</code>	Extraer	<code>filter(fecha_prestamo__year=2026)</code>
Navegación con <code>__</code>	JOIN	<code>filter(autor__nombre="Ursula")</code>

6. Parte V — Las 4 queries del TP-4 en profundidad

Esta es la sección **más importante** de toda la guía. Si entendés esto, el TP-4 sale. Cada query se explica 3 veces: qué hace, cómo funciona el ORM, qué SQL genera.

6.1. Query 1 — `libros_por_categoria(nombre_categoria)`

```
def libros_por_categoria(nombre_categoria: str):  
    return Libro.objects.filter(categorias__nombre=nombre_categoria)
```

Qué hace: devuelve todos los Libros que tienen una categoría con ese nombre.

Cómo funciona el ORM: `categorias__nombre` atraviesa:

- `categorias` → la M2M → tabla intermedia
- `nombre` → al campo `nombre` de Categoría

SQL generado:

```
SELECT "catalogo_libro".*
FROM "catalogo_libro"
INNER JOIN "catalogo_libro_categorias"
    ON ("catalogo_libro"."id" = "catalogo_libro_categorias"."libro_id")
INNER JOIN "catalogo_categoria"
    ON ("catalogo_libro_categorias"."categoria_id" = "catalogo_categoria"."id")
WHERE "catalogo_categoria"."nombre" = 'fantasía'
```

Gotcha: si un libro tiene 2 categorías que matchean, va a aparecer 2 veces. En el TP-4 no pasa, pero en casos reales agregar `.distinct()`.

6.2. Query 2 — `autores_con_mas_de_n_libros(n)`

```
from django.db.models import Count

def autores_con_mas_de_n_libros(n: int):
    return Autor.objects.annotate(
        cantidad_libros=Count("libros")
    ).filter(cantidad_libros__gt=n)
```

Qué hace: devuelve los autores que tienen más de `n` libros.

annotate paso a paso:

- Recorre cada Autor en memoria lógica.
- A cada uno le "pega" un campo extra `cantidad_libros` que es `COUNT(libros)`.
- Luego `.filter(cantidad_libros__gt=n)` filtra por ese campo anotado.

SQL generado:

```
SELECT "catalogo_autor".*, COUNT("catalogo_libro"."id") AS "cantidad_libros"
FROM "catalogo_autor"
LEFT OUTER JOIN "catalogo_libro"
    ON ("catalogo_autor"."id" = "catalogo_libro"."autor_id")
GROUP BY "catalogo_autor"."id"
HAVING COUNT("catalogo_libro"."id") > 1
```

Anti-patrón que NO hay que hacer:

```
# ❌ N+1 queries: 1 para Autor.objects.all() + N para a.libros.count()
[a for a in Autor.objects.all() if a.libros.count() > n]
```

6.3. Query 3 — `libros_sin_disponibilidad()`

```
from django.db.models import Count, Q, F

def libros_sin_disponibilidad():
    return Libro.objects.annotate(
        activos=Count(
            "prestamos",
            filter=Q(prestamos__fecha_devolucion__isnull=True)
        )
    ).filter(activos=F("cantidad_total"))
```

Qué hace: devuelve los libros donde **todos** los ejemplares están prestados (prestamos activos = cantidad_total).

Descomposición:

1. `annotate(activos=...)` — a cada libro le agrega un campo `activos` con la cuenta de préstamos con `fecha_devolucion IS NULL`.
2. El `filter=Q(...)` dentro de `count` filtra **qué préstamos** contar, no qué libros.
3. Luego `.filter(activos=F("cantidad_total"))` — `F("cantidad_total")` hace referencia a la columna de la misma fila.

¿Por qué `F()`? Porque si escribieras `filter(activos=self.cantidad_total)`, Python intentaría evaluar `self.cantidad_total` inmediatamente (no tiene sentido sin un objeto). `F()` le dice al ORM: “cuando generes el SQL, usá la columna `cantidad_total` de la misma fila”.

SQL generado:

```
SELECT "catalogo_libro".*,
       COUNT("catalogo_prestamo"."id") FILTER (
         WHERE "catalogo_prestamo"."fecha_devolucion" IS NULL
       ) AS "activos"
FROM "catalogo_libro"
LEFT OUTER JOIN "catalogo_prestamo"
  ON ("catalogo_libro"."id" = "catalogo_prestamo"."libro_id")
GROUP BY "catalogo_libro"."id"
HAVING COUNT("catalogo_prestamo"."id") FILTER (
  WHERE "catalogo_prestamo"."fecha_devolucion" IS NULL
) = "catalogo_libro"."cantidad_total"
```

Restricción del TP: resolver con ORM puro (NO con `for libro in ...: if libro.disponibles() == 0`).

6.4. Query 4 — `top_n_libros_mas_prestados(n)`

```
def top_n_libros_mas_prestados(n: int):
    return (
        Libro.objects
        .annotate(total_prestamos=Count("prestamos"))
        .order_by("-total_prestamos")[:n]
    )
```

Qué hace: los N libros con más préstamos en total (activos + devueltos).

Claves:

- `Count("prestamos")` cuenta **todos** los préstamos de cada libro (sin filtro).
- `order_by("-total_prestamos")` — `-` = DESC.
- `[:n]` — slicing se traduce a `LIMIT n` en SQL (no trae todo a Python).

SQL generado:

```
SELECT "catalogo_libro".*, COUNT("catalogo_prestamo"."id") AS "total_prestamos"
FROM "catalogo_libro"
LEFT OUTER JOIN "catalogo_prestamo"
ON ("catalogo_libro"."id" = "catalogo_prestamo"."libro_id")
GROUP BY "catalogo_libro"."id"
ORDER BY "total_prestamos" DESC
LIMIT 5
```

6.5. `annotate` vs `aggregate` — la diferencia clave

Método	Devuelve	Ejemplo
<code>annotate()</code>	Un QuerySet con una columna extra por fila	<code>Autor.objects.annotate(n=Count("libros"))</code> → 1 fila por autor + <code>n</code>
<code>aggregate()</code>	Un dict con un solo valor (toda la tabla)	<code>Libro.objects.aggregate(total=Count("id"))</code> → <code>{"total": 42}</code>

Regla: `annotate` por fila, `aggregate` para toda la query.

7. Tutorial B — Resolver el TP-4 paso a paso

Este tutorial asume que clonaste el repo del TP-4.

7.1. Setup

```
git clone <url-del-classroom> tp-4
cd tp-4

python -m venv .venv
.venv\Scripts\Activate.ps1
# o: source .venv/bin/activate

pip install -r requirements.txt

# Verificar que los tests fallan (rojo inicial)
python manage.py test -v 2
```

7.2. Implementar `models.py`

Abre `catalogo/models.py`. Reemplazá cada `pass` con el código de §5.3 de esta guía. Commit cada modelo para llegar a ≥8 commits:

```
git add catalogo/models.py
git commit -m "feat(models): implementar Autor con email único"
# ... siguiente modelo ...
git commit -m "feat(models): implementar Categoria"
# ...
git commit -m "feat(models): implementar Libro con FK y M2M + métodos de dominio"
git commit -m "feat(models): implementar Prestamo con nullable fecha_devolucion"
```

7.3. Migraciones

```
python manage.py makemigrations catalogo
python manage.py migrate

git add catalogo/migrations/
git commit -m "chore(migrations): migración inicial"
```

7.4. Tests de modelos en verde

```
python manage.py test catalogo.tests.test_models -v 2
```

Si hay rojo, leer el traceback, corregir, repetir.

7.5. Implementar `queries.py`

Abrí `catalogo/queries.py`. Reemplazá cada `raise NotImplementedError` con el código de §6 de esta guía.

```
git add catalogo/queries.py
git commit -m "feat(queries): libros_por_categoria"
# siguiente query
git commit -m "feat(queries): autores_con_mas_de_n_libros con annotate"
# ...
```

7.6. Tests completos

```
python manage.py test -v 2
```

Todo verde → push → GitHub Actions corre el autograder → nota.

7.7. Si Actions falla pero local OK

Lo más común: olvidaste commitear las migraciones.

```
git status
# Si ves catalogo/migrations/0001_initial.py en "untracked", arréglalo:
git add catalogo/migrations/
git commit -m "chore: agregar migraciones faltantes"
git push
```

8. Tutorial C — Django shell en 10 queries indispensables

Abrí `python manage.py shell` y practicá estas 10:

```

# 1. Todos los libros
>>> Libro.objects.all()

# 2. Un libro por PK
>>> Libro.objects.get(pk=1)

# 3. Filtrar por relación FK
>>> Libro.objects.filter(autor__nombre__icontains="le")

# 4. Ordenar
>>> Libro.objects.order_by("-fecha_publicacion")

# 5. Limitar
>>> Libro.objects.all()[:3]

# 6. Contar
>>> Libro.objects.filter(cantidad_total__gt=0).count()

# 7. Agregar (total toda la tabla)
>>> from django.db.models import Sum
>>> Libro.objects.aggregate(total=Sum("cantidad_total"))
{"total": 42}

# 8. Anotar (por fila)
>>> Autor.objects.annotate(n=Count("libros")).values("nombre", "n")
<QuerySet [{"nombre": "Ursula", "n": 3}, ...]>

# 9. Q (OR complejos)
>>> from django.db.models import Q
>>> Libro.objects.filter(Q(cantidad_total=0) | Q(titulo__icontains="test"))

# 10. Ver el SQL generado
>>> print(Libro.objects.filter(autor__nombre="Ursula").query)

```

9. Tutorial D — Leer errores del autograder

El autograder corre `python manage.py test` en GitHub Actions. Si falla, la pestaña **Actions** del repo muestra el log. Errores típicos:

9.1. `IntegrityError: UNIQUE constraint failed: catalogo_autor.email`

Causa: dos autores con mismo email en tus tests (o en los datos del test del autograder y tus propios datos). **Arreglo:** usar emails distintos. Si es en `setUpTestData`, revisar que no se corra dos veces.

9.2. `django.db.utils.OperationalError: no such table: catalogo_libro`

Causa: falta aplicar migraciones, o las migraciones no están commiteadas. **Arreglo local:** `python manage.py migrate`. **Arreglo para Actions:** `git add catalogo/migrations/ && git commit && git push`.

9.3. `FieldError: Cannot resolve keyword 'prestamo' into field`

Causa: usaste `prestamo` pero el `related_name` es `prestamos`. Django espera el `related_name` (o `prestamo_set` si no lo definiste). **Arreglo:** revisar el `related_name` en `models.py` y usar exactamente ese nombre en los lookups.

9.4. `TypeError: 'NotImplementedType' object is not callable`

Causa: olvidaste implementar un método / función — quedó `raise NotImplementedError`. **Arreglo:** implementarlo.

9.5. `AssertionError: 0 != 2`

Causa: el test esperaba 2 pero tu query devolvió 0. Suele ser un lookup mal tipeado o un `related_name` mal. **Arreglo:** abrir el shell, ejecutar la misma query manualmente, ver por qué no devuelve lo esperado.

10. Anti-patrones a evitar

Anti-patrón	Por qué es malo	Qué hacer
<code>for x in qs: ...</code> con queries adentro	N+1 queries	<code>annotate</code> o <code>select_related</code>
<code>len(qs)</code> para contar	Trae todo	<code>qs.count()</code>
<code>qs[0]</code> sin <code>order_by</code>	Resultado imprevisible	<code>qs.order_by(...).first()</code>
<code>get()</code> sin try/except en producción	<code>DoesNotExist</code>	Usar <code>filter(...).first()</code> o capturar
<code>Model.objects.all()</code> en templates sin límite	Rendimiento	<code>[:N]</code> o paginación
Vistas como funciones (FBVs)	Rompe convención cátedra	Usar CBVs
<code>@login_required</code>	Decorador FBV	<code>LoginRequiredMixin</code>
<code>from .models import *</code>	Imports opacos	Importar nombres explícitos

11. FAQ — preguntas más frecuentes

1. ¿Por qué CBV y no FBV si los tutoriales de la web usan FBV? Por coherencia POO con el resto de la cátedra y por reusabilidad vía mixins. Los tutoriales genéricos usan FBV porque son más cortos en texto, no porque sean mejores.

2. ¿Cuál es la diferencia entre `null=True` y `blank=True`? `null=True` permite `NULL` en la BD; `blank=True` permite vacío en formularios. Para texto: solo **blank**. Para fecha/número opcional: **ambos**.

3. ¿Cuándo `PROTECT` y cuándo `CASCADE`? Depende del dominio. Si el hijo **no debería existir** sin el padre → `CASCADE`. Si el padre **no puede desaparecer** mientras tenga hijos → `PROTECT`. Si “se pierde referencia” es aceptable → `SET_NULL`.

4. ¿ `related_name` es opcional u obligatorio? Opcional. Si no lo ponés, Django usa `<modelo>_set` . Es buena práctica ponerlo siempre para que el código sea legible.
5. ¿ `get()` vs `filter().first()` ? `get()` lanza excepciones (`DoesNotExist` , `MultipleObjectsReturned`) si no hay exactamente 1. `filter().first()` devuelve `None` . Usá `get()` cuando esperás exactamente 1 y el fallo sería un bug.
6. ¿Por qué `annotate` y no `count()` en un loop? Porque `annotate` genera 1 sola query SQL con GROUP BY; el loop hace N+1 queries. En 10 libros no se nota; en 10.000 el servidor se cae.
7. ¿Qué significa que QuerySet es “lazy”? Que el SQL no se ejecuta al construirlo, sino al momento de usarlo (iterar, slicear, contar, llamar list()). Podés armar una query compleja encadenando filtros y solo al final, cuando la usás, va a la BD.
8. `makemigrations` ¿modifica la BD? No. Solo genera un archivo Python en `migrations/` . La BD se modifica con `migrate` .
9. ¿Cómo reseteo la BD en desarrollo? Borrá `db.sqlite3` y los archivos `0*.py` en `migrations/` (dejá `__init__.py`). Luego `makemigrations` + `migrate` . Nunca hacerlo en producción.
10. El autograder falla en GitHub pero local funciona, ¿por qué? 99% de las veces: olvidaste commitear las migraciones. Verificá con `git status` que `catalogo/migrations/0001_initial.py` esté en el repo.
11. ¿Puedo usar raw SQL ? `Model.objects.raw("SELECT ...")` existe y funciona, pero el TP-4 exige ORM puro para las 4 queries. Se califica rojo si lo usás.
12. ¿Cómo veo el SQL que genera mi query? `print(qs.query)` en el shell. Sirve para debug y para aprender.
13. ¿Qué hace `F()` ? Referencia a otra columna de la misma fila sin traer el valor a Python. Usala cuando filtros o compares dos columnas entre sí.
14. ¿Qué hace `Q()` ? Permite combinar condiciones con `&` (AND), `|` (OR), `~` (NOT). Sin `Q` , los filtros de `filter()` son siempre AND.
15. ¿Cómo corro un solo test? `python manage.py test catalogo.tests.test_models.ModelsTestCase.test_disponibilidad_sin_prestamos -v 2`
-

12. Checklist final del TP-4

Antes de hacer push a main, verificá:

- ☐ [] `catalogo/models.py` sin ningún `pass` — los 4 modelos implementados.
 - ☐ [] `catalogo/queries.py` sin ningún `raise NotImplementedError` — las 4 queries implementadas.
 - ☐ [] `__str__` en los 4 modelos.
 - ☐ [] `related_name` explícito en FKs y M2M.
 - ☐ [] `python manage.py makemigrations` no genera nada nuevo (ya está todo migrado).
 - ☐ [] `python manage.py migrate` corre sin errores.
 - ☐ [] `python manage.py test -v 2` → **todos verdes localmente**.
 - ☐ [] Los archivos de migración **están commiteados** (`git status` limpio).
 - ☐ [] ≥ 8 commits con mensajes descriptivos.
 - ☐ [] Push a `main` → GitHub Actions → **verde**.
-

13. Glosario

- **App (Django):** módulo con una feature específica (models + views + urls + templates).
 - **CBV:** Class-Based View — vista implementada como clase.
 - **CRUD:** Create, Read, Update, Delete.
 - **FBV:** Function-Based View — vista como función (prohibido en la cátedra).
 - **Framework:** código que **te llama** y en el cual “llenás huecos”.
 - **HTTP:** protocolo request/response de la web.
 - **Impedance mismatch:** desajuste entre modelo OO y modelo relacional.
 - **Lazy:** una query o un valor se computa solo cuando se consume.
 - **Migración:** archivo que describe cambios de esquema de BD.
 - **MVC / MVT:** patrones que separan datos, lógica y presentación.
 - **N+1:** anti-patrón donde 1 query principal desencadena N queries extra.
 - **ORM:** Object-Relational Mapper — capa que traduce objetos ↔ tablas.
 - **QuerySet:** colección lazy de objetos Django, encadenable.
 - **related_name:** nombre del reverse accessor en relaciones.
-

14. Recursos de referencia

Toda la documentación oficial de Django 6.0 está **offline** en ChromaDB de la cátedra (colección `django-6.0-docs` , 2443 fragmentos indexados) y Python 3.14 idem (`python-3.14-docs` , 5306 fragmentos). Consultá a tu docente para cómo buscar.

- Docs Django oficiales: <https://docs.djangoproject.com/en/5.1/>
 - Tutorial oficial "Writing your first Django app": 7 partes — muy recomendado leer parte 1 y 2 antes del TP-4.
 - Queryset API reference: <https://docs.djangoproject.com/en/5.1/ref/models/queriesets/>
 - Class-based views reference: <https://docs.djangoproject.com/en/5.1/ref/class-based-views/>
-

Fin de la guía — Tema 03. — Dra. Sofía, study-guide-writer EDU, 22/04/2026.